



CORE COMPUTER SCIENCE

Structured Query Language

Placement Preparation Handbook

Created by Pranav Gawai

SQL Preparation Roadmap

PLACEMENT PREPARATION GUIDE

How to use this Handbook:

SQL questions are highly standardized in coding interviews. Master the **Logical Execution Order** of queries first. Focus heavily on **Window Functions** (ROW_NUMBER vs DENSE_RANK), **Joins** (handling nulls in outer joins), **Indexing** structures (seek vs scan), and **Transaction Isolation** anomaly models.

Three-Phase Learning Plan

PHASE 1

Fundamentals & Joins

Learn projection filtering (WHERE vs HAVING), aggregate null behaviors, and physical join mechanics (Nested Loop, Hash, Merge Joins). (Topics 1 – 14)

PHASE 2

Advanced SQL

Study correlated subqueries, recursive CTEs, and window partition frames (LEAD, LAG, DENSE_RANK). (Topics 15 – 23)

PHASE 3

Indexing, ACID & Practice

Understand B+ Trees, clustered/non-clustered indexes, transaction isolation levels, and solve classic interview queries. (Topics 24 – 40)

Table of Contents

SQL Fundamentals & Advanced Systems · Placement Preparation Handbook

01 SELECT Statement	p. 04	15 Subqueries (Nested Queries)	p. 18	29 ACID Properties	p. 32
02 WHERE Clause	p. 05	16 Correlated Subqueries	p. 19	30 Transactions	p. 33
03 ORDER BY Clause	p. 06	17 CTEs (Common Table Expressions)	p. 20	31 Locks & Concurrency	p. 34
04 DISTINCT Clause	p. 07	18 Window Functions	p. 21	32 Nth Highest Salary	p. 35
05 LIMIT & OFFSET	p. 08	19 ROW_NUMBER()	p. 22	33 Duplicate Records	p. 36
06 GROUP BY Clause	p. 09	20 RANK()	p. 23	34 Consecutive Numbers	p. 37
07 HAVING Clause	p. 10	21 DENSE_RANK()	p. 24	35 Department Top Earners	p. 38
08 Aggregate Functions	p. 11	22 LEAD()	p. 25	36 Running Total	p. 39
09 INNER JOIN	p. 12	23 LAG()	p. 26	37 Top N Per Group	p. 40
10 LEFT OUTER JOIN	p. 13	24 Database Indexing	p. 27	38 Customer Retention	p. 41
11 RIGHT OUTER JOIN	p. 14	25 Clustered Index	p. 28	39 Ranking Queries	p. 42
12 FULL OUTER JOIN	p. 15	26 Non-Clustered Index	p. 29	40 Date Queries	p. 43
13 SELF JOIN	p. 16	27 Query Optimization	p. 30		
14 CROSS JOIN	p. 17	28 Execution Plan Basics	p. 31		

SQL FUNDAMENTALS

Yield: ★★★★★

01 – SELECT Statement

Projection mechanics and relational query basis.

DEFINITION & KEY CONCEPTS

The **SELECT** statement is the foundational command of relational algebra's projection (π) operation, specifying which columns to retrieve from a table. Instead of selecting all columns (*), best practice mandates selecting explicit columns to minimize network payload, disk I/O, and optimize query caches.

Projection Flow

Table [A, B, C, D] (Disk Scan)

↓ SELECT A, C

Result [A, C] (Memory Projection)

WHERE USED IN INDUSTRY

Data warehouse layers (e.g. Snowflake, BigQuery) charge queries by volume of scanned bytes; selecting explicit columns is critical to contain cloud bills.

INTERVIEW DEPTH

- L1:** Logical projection vs physical page scanning steps.
- L2:** Covering indexes and column pruning optimization.
- L3:** Impact of **SELECT *** on network payload and packet latency.
- L4:** Metadata catalog lock contention during query parsing.
- L5:** Memory buffer allocation adjustments based on row width.

CORE INTERVIEW QUESTION

"Why is using **SELECT *** considered an anti-pattern in production environments?"

KEYWORDS TO DROP

SCHEMA BINDING

NETWORK PAYLOAD

COVERING INDEX

DISK I/O

MODEL ANSWER

"Using **SELECT *** pulls unnecessary columns, increasing memory and network transit times. It also prevents the query optimizer from leveraging covering indexes (where all requested columns reside in the index). Furthermore, schema changes (e.g., adding or reordering columns) can break application integrations that rely on explicit ordinal column bindings."

HOW TO EXPLAIN IN INTERVIEW

"Always projection first: Using **SELECT *** pulls all fields from the page tables. In production systems, we use explicit column selection to minimize database disk page loads and cache pollution, allowing covering index scans."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming **SELECT *** has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand ***** to all columns.

INTERVIEWER TRAP

Claiming **SELECT *** has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand ***** to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

SQL FUNDAMENTALS

Yield: ★★★★★

02 – WHERE Clause

Row-level filtering and sargable predicates.

DEFINITION & KEY CONCEPTS

The `WHERE` clause implements relational selection (σ), filtering rows dynamically *before* any grouping or aggregation takes place. To execute efficiently, filter predicates should be 'SARGable' (Search Argument Able), enabling the query optimizer to utilize index seeks rather than full table scans.

Predicate	SARGable?	Action
<code>age = 30</code>	Yes	Index Seek
<code>YEAR(date) = 2026</code>	No	Full Table Scan
<code>date >= '2026-01-01'</code>	Yes	Index Range Seek

WHERE USED IN INDUSTRY

Payment backends index checking transaction states. SARGable range indexes filter active transactions in microseconds during checkout operations.

INTERVIEW DEPTH

- L1: Selection operations on relational tables.
- L2: SARGable predicates and index scan/seek selections.
- L3: Implicit type conversions and indexing degradation.
- L4: Logical operator precedence (AND before OR evaluation).
- L5: Index range scan operations on compound indexes.

CORE INTERVIEW QUESTION

"What is a SARGable predicate, and why does wrapping an indexed column in a function like `LOWER(email) = 'abc'` degrade performance?"

SARGABLE

INDEX SEEK

TABLE SCAN

FUNCTION WRAP

MODEL ANSWER

"A SARGable predicate allows the database engine to perform an index seek. Wrapping an indexed column in a function (like `LOWER()` or `YEAR()`) prevents the optimizer from using the index tree directly, as it must calculate the function's output for every single row, forcing a full table scan."

REAL INTERVIEW SCENARIO

Interviewer: "Why did my index seek break when I filtered with `WHERE SUBSTRING(phone, 1, 3) = '+1'`?"

Candidate: "Because wrapping phone in a function makes it non-SARGable. The database must scan every record's phone number to calculate the substring. Use `WHERE phone LIKE '+1%'` instead."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividh and neaunnt

SQL FUNDAMENTALS

Yield: ★★★★★

03 – ORDER BY Clause

Sorting mechanics and database sort buffers.

DEFINITION & KEY CONCEPTS

The `ORDER BY` clause sorts result sets logically. Relational databases do not guarantee any order by default (data is returned as an unordered bag of rows). Sorting requires processing power; it either reads records directly from an ordered index or uses database Sort Buffers (filesort) in memory.

Sort Execution Paths:

- **Index Scan:** $O(N)$ reading directly from sorted index leaf nodes.
- **File Sort:** $O(N \log N)$ using sorting algorithms in memory/temp files.



WHERE USED IN INDUSTRY

E-commerce product grids sorted by price rely on indexed fields to retrieve items instantly, bypassing memory sorting limits.



INTERVIEW DEPTH

- L1:** Implicit sorting behaviors on relational schemas.
- L2:** Index-based sorting vs filesort (Sort Buffers).
- L3:** Quicksort algorithms in memory vs external merge sorts.
- L4:** Sort buffer size configuration and temp file degradation.
- L5:** Ordering NULL values: NULLS FIRST vs NULLS LAST.

CORE INTERVIEW QUESTION

"How does the database execute an `ORDER BY` query when no index is present, and what is a filesort?"

FILESORT

SORT BUFFER

TEMPORARY TABLE

INDEX SCAN

MODEL ANSWER

"When no sorted index is available, the engine performs a 'filesort'. It copies matching records to a memory buffer (Sort Buffer) and applies quicksort. If the data size exceeds the sort buffer capacity, it splits the data into temporary disk files, creating high I/O latency."



HOW TO EXPLAIN IN INTERVIEW

"**Sorting isn't free:** Relational tables are unordered. Sorting uses temporary disk files (filesort) if the sort buffer runs out of space. Creating an index on the sorting column lets the database read records in order at $O(1)$ sort cost."

⚠ COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

🔍 TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

🔴 INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

💡 MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

SQL FUNDAMENTALS

Yield: ★★★★★

04 – DISTINCT Clause

Deduplication and grouping execution paths.

DEFINITION & KEY CONCEPTS

The **DISTINCT** clause filters out duplicate rows, returning only unique values or combinations. Under the hood, databases implement deduplication using either an **Index Scan** (fastest), a **Hash Table aggregation** (hash aggregate), or by **sorting and merging** the result set.

Deduplication Logic

Rows [1, 2, 2, 3] → Hash Table Lookup → Unique [1, 2, 3]



WHERE USED IN INDUSTRY

User analytics aggregates identify unique customer IDs accessing platform APIs monthly to calculate billing cohorts.



INTERVIEW DEPTH

- L1:** Deduplication basics and unique row values.
- L2:** Deduplication mechanisms: Sort vs Hash Aggregates.
- L3:** **DISTINCT** vs **GROUP BY** compiler translation structures.
- L4:** Distinct scans using index prefix trees.
- L5:** Memory footprint of distinct checks on huge datasets.

CORE INTERVIEW QUESTION

"What is the physical execution difference between **SELECT DISTINCT col** and **SELECT col ... GROUP BY col**?"

HASH AGGREGATE

GROUP DEDUPLICATION

SORT MERGE

EXECUTION PLAN

MODEL ANSWER

"For simple deduplication, modern query optimizers translate both **DISTINCT** and **GROUP BY** into the same physical operators (usually Hash Aggregates). However, **DISTINCT** is logically used to remove duplicates from columns, while **GROUP BY** is intended to partition rows for aggregate functions."



REAL INTERVIEW SCENARIO

Interviewer: "Is **DISTINCT** always slow?"

Candidate: "No. If columns are indexed, the database uses a fast Index Scan. If not, it uses a Hash Aggregate in memory. It only slows down when memory limits force a Sort Merge to disk."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming **SELECT *** has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand ***** to all columns.

INTERVIEWER TRAP

Claiming **SELECT *** has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand ***** to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

SQL FUNDAMENTALS

Yield: ★★★★★

05 – LIMIT & OFFSET

Paging strategies and pagination limits.

DEFINITION & KEY CONCEPTS

The `LIMIT` (or `TOP / FETCH FIRST`) clause limits the number of rows returned. `OFFSET` skips a specified number of rows. While ideal for frontend paging, high `OFFSET` queries degrade performance since the database must scan and discard all skipped rows.

Offsets Performance Cost:

- `OFFSET 10 LIMIT 10`: Scans 20 rows (Fast)
- `OFFSET 10000 LIMIT 10`: Scans 10,010 rows, discards 10,000 (Slow)

WHERE USED IN INDUSTRY

Mobile app dashboards leverage keyset seeking (cursor paging) to fetch activity updates, skipping slow offsets under high traffic.

INTERVIEW DEPTH

- L1:** Limiting row outputs and page offset queries.
- L2:** Offset scan-and-discard overheads on high pages.
- L3:** Keyset pagination seeking (Direct Index Seek).
- L4:** Non-deterministic `LIMIT` queries without `ORDER BY`.
- L5:** Optimizer early termination paths during limit matching.

CORE INTERVIEW QUESTION

"Why does database performance degrade as `OFFSET` values increase, and what is Keyset Pagination?"

KEYSET PAGINATION

ROW DISCARD

OFFSET PAGING

SEEK METHOD

MODEL ANSWER

"With high offsets, the engine must read all skipped rows from disk and discard them. Keyset Pagination (or cursor paging) avoids this by filtering on the last-seen identifier (e.g., `WHERE id > last_seen_id LIMIT 10`), performing a direct index seek instead."

HOW TO EXPLAIN IN INTERVIEW

"**Cursor Paging:** Instead of `LIMIT 10 OFFSET 10000`, filter with `WHERE id > last_seen_id LIMIT 10`. This avoids scanning and discarding the first 10,000 records, executing in microseconds."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

FILTERING & AGGREGATION

Yield: ★★★★★

06 – GROUP BY Clause

Aggregation boundaries and hashing execution.

DEFINITION & KEY CONCEPTS

The `GROUP BY` clause partitions a result set into groups. The database groups rows using either a **Hash Aggregate** (building an in-memory hash table of unique group keys) or a **Sort Aggregate** (sorting data by keys and summarizing consecutive matching records).

Hash Aggregation Path:

Key "A" → Hash Index [0] (Sum = 12)
Key "B" → Hash Index [1] (Sum = 45)



WHERE USED IN INDUSTRY

Reporting engines group server telemetry data by hour and region to visualize operational trends on Grafana dashboards.



INTERVIEW DEPTH

- L1:** Collapsing duplicate columns into group keys.
- L2:** Hash Aggregation vs Sort Aggregation physical paths.
- L3:** Single-column vs multi-column grouping structures.
- L4:** Indeterminacy and the functional dependency validation.
- L5:** Buffer pool optimizations during heavy grouping operations.

CORE INTERVIEW QUESTION

"Why can't you select non-aggregated columns in a query that utilizes the `GROUP BY` clause?"

FUNCTIONAL DEPENDENCY

SQL STANDARD

HASH TABLE KEY

INDETERMINACY

MODEL ANSWER

"Selecting non-aggregated columns that are not part of the `GROUP BY` clause violates relational logic. Because multiple rows are collapsed into a single row, the database cannot determine which value of the ungrouped column to display, leading to syntax errors."



REAL INTERVIEW SCENARIO

Interviewer: "Why did this fail: `SELECT department, AVG(salary) FROM emp GROUP BY office?`"

Candidate: "Because `department` is not in the `GROUP BY`. Multiple `department` values exist inside each office group; the database cannot collapse them without an aggregate function."

⚠ COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

🔍 TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

🚫 INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.



MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

FILTERING & AGGREGATION

Yield: ★★★★★

07 – HAVING Clause

Group-level filters vs row-level filters.

DEFINITION & KEY CONCEPTS

The **HAVING** clause filters aggregated groups. Unlike the **WHERE** clause (which filters rows prior to group generation), **HAVING** acts on grouped results after aggregation. Mixing their logic can severely degrade performance by delaying filter operations.

Raw Rows

↓ **WHERE** (Filters raw rows)

GROUP BY (Aggregates)

↓ **HAVING** (Filters aggregated keys)

Final Groups



WHERE USED IN INDUSTRY

SaaS analytics modules filter user groups that averaged more than 50 API errors daily to alert developer integration teams.



INTERVIEW DEPTH

- L1:** Filtering aggregated group outcomes.
- L2:** **WHERE** execution order vs **HAVING** execution order.
- L3:** Delaying evaluations: Non-aggregate filters in **HAVING**.
- L4:** Optimizer push-down of **HAVING** filters to **WHERE**.
- L5:** Complex aggregates inside **HAVING** condition expressions.

CORE INTERVIEW QUESTION

"What is the core difference between **WHERE** and **HAVING**, and which runs first?"

LOGICAL EXECUTION

POST-AGGREGATION

PRE-AGGREGATION

PERFORMANCE OPTIMIZATION

MODEL ANSWER

"**WHERE** filters individual records before groups are formed, whereas **HAVING** filters group results after aggregation. **WHERE** is executed first. Filtering records in the **WHERE** clause is always preferred because it reduces the volume of rows that must be aggregated."



HOW TO EXPLAIN IN INTERVIEW

"Filter Early: Never filter raw row conditions in **HAVING**. Write row filters in **WHERE** first to reduce the rows that must be aggregated, saving CPU and sort buffers."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming **SELECT *** has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand ***** to all columns.

INTERVIEWER TRAP

Claiming **SELECT *** has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand ***** to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

FILTERING & AGGREGATION

Yield: ★★★★★

08 – Aggregate Functions

SUM, AVG, COUNT, MIN, MAX and NULL behaviors.

DEFINITION & KEY CONCEPTS

Aggregate functions collapse columns of values into single scalar metrics. A critical aspect of interview evaluations is understanding how these functions handle `NULL` values. Standard mathematical aggregates discard `NULL`s, whereas `COUNT(\*)` counts complete row occurrences.

Salary Col	COUNT(*)	COUNT(Salary)	AVG(Salary)
1000	3	2	1500 (3000/2)
NULL			
2000			

WHERE USED IN INDUSTRY

Risk valuation modules average portfolio returns, utilizing COALESCE checks to map missing stock entries to default values.

INTERVIEW DEPTH

- L1: SUM, AVG, COUNT, MIN, MAX definitions.
- L2: COUNT(*) row count vs COUNT(col) value count.
- L3: Aggregate behaviors with NULL (Zero vs Null exclusion).
- L4: Using COALESCE to force default numerical aggregates.
- L5: Custom aggregate functions using procedural database scripts.

CORE INTERVIEW QUESTION

"How do `COUNT(*)`, `COUNT(col)`, and `AVG(col)` differ in handling `NULL` values?"

NULL ELIMINATION

ROW COUNT

AGGREGATE DENOMINATOR

COALESCE

MODEL ANSWER

"`COUNT(*)` counts every row regardless of column content (including `NULL` rows). `COUNT(col)` counts only rows where the specified column is not `NULL`. `AVG(col)` calculates the sum of non-`NULL` values divided by the count of non-`NULL` values. To treat `NULL` as zero in averages, wrap the column in `COALESCE(col, 0)`."

REAL INTERVIEW SCENARIO

Interviewer: "What is the average of [10, NULL, 20]?"

Candidate: "In SQL, `AVG()` ignores `NULL`, yielding 15 (30/2). To count `NULL` as 0, write `AVG(COALESCE(col, 0))`, which yields 10 (30/3)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

JOINS

Yield: ★★★★★

09 – INNER JOIN

Intersection operations and physical join algorithms.

DEFINITION & KEY CONCEPTS

An **INNER JOIN** returns rows that match join conditions on both tables, representing a mathematical intersection. Physically, database engines use three core join algorithms: **Nested Loop** (best for small tables), **Hash Join** (best for large unsorted tables), and **Merge Join** (best for sorted indexed relations).

Physical Join Algorithms:

- **Nested Loop:** Outer row scans inner table index.
- **Hash Join:** Build hash table for Table A, probe with B.
- **Merge Join:** Line up and scan two pre-sorted index lists.



WHERE USED IN INDUSTRY

Order processing modules query orders joined with client emails to dispatch tracking status emails during warehouse releases.



INTERVIEW DEPTH

- L1:** Mathematical intersection of two relation schemas.
- L2:** Physical join algorithms: Nested Loop vs Hash vs Merge.
- L3:** Build vs Probe phases in Hash Join execution.
- L4:** Logical join reordering by cost optimizers.
- L5:** Hash table bucket collisions during join calculations.

CORE INTERVIEW QUESTION

"Explain how a Hash Join works and under what conditions the engine prefers it over a Nested Loop Join."

HASH JOIN

PROBE PHASE

NESTED LOOP

BUILD PHASE

MODEL ANSWER

"A Hash Join operates in two phases: **Build Phase** (reads the smaller table and constructs a hash table of join keys in memory) and **Probe Phase** (scans the larger table, hashing its join keys to look for matches). The engine prefers it over Nested Loop for large, unsorted datasets where indexing is absent."



HOW TO EXPLAIN IN INTERVIEW

Join Algorithms: Databases use Hash Joins for large tables, Merge Joins for pre-sorted indexed lists, and Nested Loops for small lookups. The optimizer calculates data volumes to pick the best physical path."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.



MEMORY TRICK

Alance sunlinith mar handividh and neaunnt

JOINS

Yield: ★★★★★

10 – LEFT OUTER JOIN

Preserving left relation records with null extensions.

DEFINITION & KEY CONCEPTS

A **LEFT OUTER JOIN** preserves all rows of the left-hand table. If a row has a matching record in the right-hand table, columns are combined; otherwise, the right-hand columns are padded with `NULL`. This is primary for identifying unmatched records (anti-patterns).

```
Left Table [A, B] LEFT JOIN Right Table [B, C]
→ Output: A (Right = NULL), B (Right = B)
```



WHERE USED IN INDUSTRY

Marketing directories audit active users, executing **LEFT JOINs** to isolate accounts that have never completed a registration checkout.



INTERVIEW DEPTH

- L1:** Preserving left-side relations with null extensions.
- L2:** Anti-Join pattern implementations to find missing links.
- L3:** Placing filters: **ON** clause vs **WHERE** clause.
- L4:** Implicit conversion of **LEFT JOIN** to **INNER JOIN**.
- L5:** Optimizing left joins over partitioned partition keys.

CORE INTERVIEW QUESTION

"How can you use a **LEFT JOIN** to find records in Table A that have no matching records in Table B, and what is this pattern called?"

ANTI-JOIN

NULL FILTERING

OUTER JOIN EXCLUSION

EXECUTION SCAN

MODEL ANSWER

"This is called an **Anti-Join**. It is implemented by left-joining Table A with Table B and filtering for non-matches in the **WHERE** clause: `SELECT A.* FROM A LEFT JOIN B ON A.id = B.id WHERE B.id IS NULL`. The optimizer filters rows where B's columns resolve to **NULL**."



REAL INTERVIEW SCENARIO

Interviewer: "Why did my **LEFT JOIN** act as an **INNER JOIN**?"

Candidate: "You placed a filter on the right table inside the **WHERE** clause (e.g. `WHERE R.status = 'A'`). Unmatched rows with **NULLs** fail this check, filtering them out. Place the filter in the **ON** clause."

⚠ COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

🔍 TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

🔴 INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

💡 MEMORY TRICK

Alance sunlinith mar handividh and neaunnt

JOINS

Yield: ★★★★★

11 – RIGHT OUTER JOIN

Preserving right relation records and query compiler transformations.

DEFINITION & KEY CONCEPTS

A `RIGHT OUTER JOIN` functions exactly like a `LEFT JOIN`, but preserves all records of the right-hand table instead. In practice, query compilers instantly convert all `RIGHT JOIN` structures into equivalent `LEFT JOIN` syntax under the hood to standardize execution planning.

Compiler Rewrite:

```
SELECT * FROM A RIGHT JOIN B ON A.id = B.id  
Is compiled as:  
SELECT * FROM B LEFT JOIN A ON B.id = A.id
```



WHERE USED IN INDUSTRY

Data ingestion pipes utilize equivalent `LEFT JOIN` rewrites to verify raw supplier records match corporate database formats.



INTERVIEW DEPTH

- L1:** Preserving right-side relations with null extensions.
- L2:** Logical equivalence of `RIGHT JOIN` to `LEFT JOIN`.
- L3:** Query parser rewrite of right joins to left joins.
- L4:** Readability constraints in complex query joins.
- L5:** Compiler parsing trees for multi-table outer joins.

CORE INTERVIEW QUESTION

"Why are `RIGHT OUTER JOIN` statements rarely written in production-grade SQL code?"

READABILITY

LEFT-TO-RIGHT FLOW

OPTIMIZER REWRITE

EQUIVALENCE

MODEL ANSWER

"Because humans read text left-to-right, writing `LEFT JOIN` queries matches standard logical processing workflows. `RIGHT JOIN` queries make complex multi-join trees hard to read, and because the optimizer compiles them to `LEFT JOIN` anyway, they provide zero performance advantage."



HOW TO EXPLAIN IN INTERVIEW

"Left Preference: Standardize on `LEFT JOIN` to read queries consistently. Planners automatically swap tables to put the smaller table in the build phase of a Hash Join anyway."

⚠️ COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

🔍 TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

🔴 INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

💡 MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

JOINS

Yield: ★★★★★

12 – FULL OUTER JOIN

Union joins and matching null padding.

DEFINITION & KEY CONCEPTS

A **FULL OUTER JOIN** (or **FULL JOIN**) returns all records from both tables. When a match exists, rows are combined; when no match exists, columns of the unmatched table are populated with `NULL`. It represents the mathematical union of two datasets.

Full Join Logic

Table A [1, 2] **FULL JOIN** Table B [2, 3] → Results
[1 (Null), 2 (2), Null (3)]



WHERE USED IN INDUSTRY

Inventory audits combine store stock lists with online catalog systems, identifying discrepancies on both physical and digital sides.



INTERVIEW DEPTH

- L1:** Union joins and complete null padding structures.
- L2:** Emulating full joins using **LEFT/RIGHT JOIN** and **UNION**.
- L3:** Performance difference: **UNION** vs **UNION ALL** emulation.
- L4:** Full join hash execution plans and memory allocation.
- L5:** Distributed full join operations in clustered databases.

CORE INTERVIEW QUESTION

"How do you emulate a **FULL OUTER JOIN** in database engines (like MySQL) that do not support it natively?"

UNION OPERATOR

LEFT JOIN

RIGHT JOIN

DEDUPLICATION

MODEL ANSWER

"In MySQL, you emulate a **FULL OUTER JOIN** by executing a **LEFT JOIN** and a **RIGHT JOIN** on the tables, and merging the two result sets using the **UNION** operator. **UNION** automatically filters out the duplicate matching records generated by both joins."

Full Join Emulation Syntax:

```
SELECT * FROM A LEFT JOIN B ON A.id = B.id
UNION
SELECT * FROM A RIGHT JOIN B ON A.id = B.id;
```

⚠ COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

🔍 TOP FOLLOW-UP

Claiming **SELECT *** has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand ***** to all columns.

🚫 INTERVIEWER TRAP

Claiming **SELECT *** has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand ***** to all columns.

💡 MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

JOINS

Yield: ★★★★★

13 – SELF JOIN

Recursive mappings and alias resolution.

DEFINITION & KEY CONCEPTS

A **SELF JOIN** is a standard join query that joins a table to itself. This is not a separate relational operator; rather, it is a technique used to map hierarchical or recursive relationships (like Manager–Employee or Parent–Child nodes) within a single table.

Hierarchical Schema Mapping:

Employee Table (emp_id, name, manager_id)

• Join: `emp E JOIN emp M ON E.manager_id = M.emp_id`

WHERE USED IN INDUSTRY

Identity management databases join employees with managers to map supervisor chains inside unified LDAP directories.

INTERVIEW DEPTH

- L1:** Joining a table schema to its own records.
- L2:** Namespace separation using clear table aliases.
- L3:** Hierarchical mappings (Parent–Child relationships).
- L4:** Performance overhead of scanning the same table twice.
- L5:** Converting self-joins to window functions or CTE paths.

CORE INTERVIEW QUESTION

"Why are table aliases mandatory when performing a **SELF JOIN**, and what happens if you omit them?"

TABLE ALIAS

AMBIGUITY

NAMESPACE CONFLICT

SELF-REFERENCE

MODEL ANSWER

"Table aliases are mandatory to resolve column name ambiguity. Without aliases, the database engine cannot distinguish which copy of the table a column belongs to, resulting in namespace conflicts and parsing errors."

HOW TO EXPLAIN IN INTERVIEW

"Scoping Aliases: When joining a table to itself, you are creating two logical views in memory. You must assign distinct alias names to each to let the SQL compiler resolve column namespaces."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

JOINS

Yield: ★★★★★

14 – CROSS JOIN

Cartesian products and multi-dimension matrix setups.

DEFINITION & KEY CONCEPTS

A `CROSS JOIN` returns the Cartesian product of the joined tables, pairing every row of the first table with every row of the second. The size of the resulting set is multiplicative (Table A rows × Table B rows). It is used to generate permutations or test matrices.

Table A (3 rows) `CROSS JOIN` Table B (5 rows)
→ Output size: $3 \times 5 = 15$ rows



WHERE USED IN INDUSTRY

Testing utilities execute `CROSS JOINs` to pair every test category with every browser environment to build verification grids.



INTERVIEW DEPTH

- L1:** Cartesian product computations and grid arrays.
- L2:** Accidental cross joins due to missing `ON` conditions.
- L3:** Permutation generation logic for combinatorics.
- L4:** Memory footprint of massive Cartesian result sets.
- L5:** Optimizer short-circuit checks for empty table pairs.

CORE INTERVIEW QUESTION

"What is the danger of executing an unintended `CROSS JOIN` on large production databases, and how does it happen?"

CARTESIAN EXPLOSION

MISSING JOIN CRITERIA

MEMORY THRASHING

TEMP SPACE EXHAUSTION

MODEL ANSWER

"Unintended Cartesian products occur when join criteria (the `ON` clause) are omitted. Joining a 10,000-row table with a 100,000-row table produces 1,000,000,000 rows. This 'Cartesian explosion' triggers out-of-memory errors and exhausts temp tablespace storage."

Permutation Matrix Example:

```
SELECT Color, Size
FROM Colors CROSS JOIN Sizes;
Returns all possible variations of colors and sizes.
```

⚠ COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

🔍 TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

🔴 INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

💡 MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

SUBQUERIES

Yield: ★★★★★

15 – Subqueries (Nested Queries)

Nested expressions and logical evaluation zones.

DEFINITION & KEY CONCEPTS

A subquery is a nested `SELECT` statement. Subqueries can be: **Scalar** (returns one value), **Row** (returns one record), or **Table** (returns multiple rows). They can be nested in `SELECT`, `FROM`, `WHERE`, or `HAVING` clauses.

Subquery Classifications:

- **Scalar:** `SELECT (SELECT MAX(price) FROM P) FROM T`
- **Filter Table:** `WHERE id IN (SELECT id FROM Active)`

WHERE USED IN INDUSTRY

Content portals filter active publications, pulling only articles whose category matches current dashboard layout arrays.

INTERVIEW DEPTH

- L1:** Scalar, row, and table nested query paths.
- L2:** `IN` vs `EXISTS` query processing and optimizations.
- L3:** Derived tables and inline query parser definitions.
- L4:** Scope visibility rules in nested `SELECT` statements.
- L5:** Optimizer flattening of basic nested subqueries.

CORE INTERVIEW QUESTION

"What is the difference between a subquery nested in the `FROM` clause vs one in the `WHERE` clause?"

DERIVED TABLE

INLINE VIEW

PREDICATE FILTER

OPTIMIZER SCOPE

MODEL ANSWER

"A subquery in the `FROM` clause acts as a **Derived Table** (or inline view). It must be assigned an alias and is treated as a temporary relation. A subquery in the `WHERE` clause acts as a **Predicate Filter** (evaluating conditions row-by-row to filter outer table rows)."

HOW TO EXPLAIN IN INTERVIEW

"Subquery Scopes: Scalar subqueries return a single cell value; table subqueries return a list for `IN` checks. Derived tables in the `FROM` clause require aliases because they function as virtual tables."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

SUBQUERIES

Yield: ★★★★★

16 – Correlated Subqueries

Row-by-row dependency loops and performance issues.

DEFINITION & KEY CONCEPTS

A correlated subquery is a nested query that references one or more columns from the outer query. Unlike independent subqueries (which execute once), a correlated subquery must theoretically execute once for every single candidate row processed by the outer query.

Correlated Loop Mechanism:

For each Outer Row (R):

Evaluate Inner Query using R.col

If True → Retain R; else Discard R.

- Complexity: $O(\text{Outer_Rows} \times \text{Inner_Rows})$

WHERE USED IN INDUSTRY

Audit systems check for suspicious ledger updates. Optimizers attempt to rewrite these to Joins to prevent looping query delays.

INTERVIEW DEPTH

- L1: Outer table references within nested queries.
- L2: Row-by-row correlation loops ($O(N \times M)$ complexity).
- L3: Optimizer decorrelation paths and join conversions.
- L4: EXISTS short-circuiting mechanics (first match escape).
- L5: Comparing correlated subqueries vs partitioned window views.

CORE INTERVIEW QUESTION

"Why are correlated subqueries generally avoided in high-performance production databases, and what is the alternative?"

ROW-BY-ROW LOOP

 $O(N^2)$ COMPLEXITY

QUERY DECORRELATION

JOIN / CTE REWRITE

MODEL ANSWER

"Correlated subqueries create $O(N \times M)$ execution paths due to row-by-row outer dependencies. Query optimizers attempt to 'decorrelate' them, but complex queries fail to optimize. Re-writing them using Joins, CTEs, or window functions allows the engine to perform set-based index scans."

Correlated Subquery Example:

```
SELECT * FROM Emp E
WHERE salary > (SELECT AVG(salary)
FROM Emp WHERE dept = E.dept);
```

Evaluates average salaries per department for each employee.

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividh and neaunnt

ADVANCED SQL

Yield: ★★★★★

17 – CTEs (Common Table Expressions)

Logical query modularity and recursive CTE execution.

DEFINITION & KEY CONCEPTS

A Common Table Expression (CTE) is a temporary result set defined using the `WITH` clause. CTEs improve readability by breaking complex logic into modular blocks. Special ****Recursive CTEs**** referencing themselves are used to traverse trees and graphs.

Recursive CTE Anchor/Recursive Union:

```
WITH RECURSIVE Hierarchy AS (  
  SELECT Anchor_Row (Base Case)  
  UNION ALL  
  SELECT Recursive_Rows JOIN Hierarchy  
)
```

WHERE USED IN INDUSTRY

Financial reporting queries divide multi-table ledger parsing stages into modular CTE blocks to keep query code readable and audit-safe.

INTERVIEW DEPTH

- L1:** Temporary result sets using `WITH` statements.
- L2:** CTE optimization: Inline view optimization vs Materialization.
- L3:** Recursive CTEs (Anchor vs Recursive union steps).
- L4:** Traversal limits (`MAXRECURSION` configurations).
- L5:** CTE scoping limits in complex transaction blocks.

CORE INTERVIEW QUESTION

"What is the execution plan difference between a CTE and a Temp Table, and are CTEs physically materialized?"

INLINE OPTIMIZATION

MATERIALIZATION

TEMP TABLE

QUERY SCOPE

MODEL ANSWER

"CTEs exist only during query compilation and are optimized inline by modern query planners (treated like virtual views, usually not materialized). Temporary Tables physically write data to disk/temp tablespaces, support indexing, and are optimal for massive datasets that require repeated queries."

HOW TO EXPLAIN IN INTERVIEW

"CTE Scoping: CTEs act as logical documentation within complex queries. In recursive CTEs, the recursive step executes until it returns an empty result set (base case exit)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

ADVANCED SQL

Yield: ★★★★★

18 – Window Functions

Calculations over table partitions and the OVER clause.

DEFINITION & KEY CONCEPTS

Window functions perform calculations across a set of table rows that are logically related to the current row. Unlike `GROUP BY` (which collapses rows), window functions preserve the identity of each row while returning aggregate or ranking results in new columns.

Window Syntax Breakdown:

```
SUM(val) OVER (PARTITION BY dept ORDER BY date)
```

- **PARTITION BY:** Group boundary lines.
- **ORDER BY:** Running frame calculation order.



WHERE USED IN INDUSTRY

Stock tracking dashboards execute window functions over partitions to compare current pricing ticks with historical daily records.



INTERVIEW DEPTH

- L1:** Window functions and the OVER clause mechanics.
- L2:** PARTITION BY vs GROUP BY (preserve vs collapse rows).
- L3:** Window frame clauses (ROWS vs RANGE specifications).
- L4:** Sorting cost overheads inside window execution plans.
- L5:** Index utilization in partition-ranking queries.

CORE INTERVIEW QUESTION

"Explain the physical difference between grouping data with **GROUP BY** and executing a window function with the **OVER** clause."

ROW IDENTITY

PARTITIONING

REDUCTION

WINDOW FRAME

MODEL ANSWER

"GROUP BY collapses multiple rows into a single summary row, discarding detail fields. A window function processes groups of rows ('windows') but leaves row identities intact, outputting the aggregate calculation on every single row."

Window Aggregation Example:

```
SELECT val,  
       SUM(val) OVER (ORDER BY id) AS running_tot  
FROM Ledger;  
Preserves row detail lines while computing cumulative summaries.
```

⚠ COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

💡 TOP FOLLOW-UP

Claiming SELECT * has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand * to all columns.

🔴 INTERVIEWER TRAP

Claiming SELECT * has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand * to all columns.

💡 MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

ADVANCED SQL

Yield: ★★★★★

19 – ROW_NUMBER()

Sequential row numbering without ties.

DEFINITION & KEY CONCEPTS

`ROW_NUMBER()` assigns a unique, sequential integer to each row within a partition. If multiple rows have identical values in the `ORDER BY` column, `ROW_NUMBER()` still assigns distinct sequential numbers arbitrarily (non-deterministic tie resolution).

Value	ROW_NUMBER()
100	1
100 (Tie)	2
200	3

WHERE USED IN INDUSTRY

Message streaming logs use `ROW_NUMBER()` grouping by ID to isolate and delete duplicate message instances from queue tables.

INTERVIEW DEPTH

- L1:** Sequential row numbering inside group partitions.
- L2:** Non-deterministic tie handling in `ROW_NUMBER()`.
- L3:** Creating deterministic sequences with secondary sort keys.
- L4:** Duplicate record isolation using `ROW_NUMBER()` filters.
- L5:** Performance difference: `ROW_NUMBER()` vs `RANK()` execution.

CORE INTERVIEW QUESTION

"How does `ROW_NUMBER()` resolve duplicate values in its sorting criteria, and how do you make it deterministic?"

DETERMINISM

TIE RESOLUTION

SEQUENTIAL ORDER

UNIQUE SORT KEY

MODEL ANSWER

"`ROW_NUMBER()` is non-deterministic for tie values—the engine assigns sequence numbers based on physical read orders. To make it deterministic, you must add a tie-breaking column to the sorting clause (e.g., `ORDER BY score DESC, unique_id ASC`)."

HOW TO EXPLAIN IN INTERVIEW

"Set-Based Logic: SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

ADVANCED SQL

Yield: ★★★★★

20 – RANK()

Ranking with ties and sequence gaps.

DEFINITION & KEY CONCEPTS

`RANK()` calculates the rank of each row within a partition. When identical sort values (ties) are encountered, they receive the same rank. However, the function leaves gaps in the sequential ranking values, reflecting the count of duplicated ties.

Score	RANK() Rank
100	1
100 (Tie)	1
90	3 (Gap left)



WHERE USED IN INDUSTRY

University grading systems assign percentile rankings to scores, leaving sequence gaps where students share tied results.



INTERVIEW DEPTH

- L1:** Ranking values within partition boundaries.
- L2:** Ties handling: identical ranks and sequence gaps.
- L3:** Ranking calculation: 1 + preceding higher rows.
- L4:** Index scan paths during multi-row ranking.
- L5:** Using `RANK()` to identify top-tier entities.

CORE INTERVIEW QUESTION

"Why does the `RANK()` function leave gaps in its sequence when ties are found, and how is the gap calculated?"

TIE GAPS

NON-CONTINUOUS

ORDINAL RANK

WINDOW ORDERING

"`RANK()` leaves gaps to preserve the cardinal position of rows. The rank of a row is calculated as 1 + (number of rows preceding it that rank higher). If two rows share rank 1, the third row has 2 rows preceding it, resulting in rank 3."



HOW TO EXPLAIN IN INTERVIEW

"Set-Based Logic: SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.



MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

ADVANCED SQL

Yield: ★★★★★

21 – DENSE_RANK()

Continuous ranking without sequence gaps.

DEFINITION & KEY CONCEPTS

`DENSE_RANK()` assigns a rank value to each row within a partition. Like `RANK()`, ties receive the same rank value. However, the sequence is contiguous—no ranks are skipped, and the rank increments by exactly 1 for the next distinct value.

Score	DENSE_RANK()
100	1
100 (Tie)	1
90	2 (No gap)

WHERE USED IN INDUSTRY

Global gaming leaderboards calculate continuous player ranks. If scores tie, subsequent ranks increment by exactly one.

INTERVIEW DEPTH

- L1: Contiguous ranking inside partition boundaries.
- L2: `DENSE_RANK()` vs `RANK()` gap comparison.
- L3: Handling duplicates without skipping ranking steps.
- L4: `DENSE_RANK()` OVER (PARTITION BY ... ORDER BY ...) syntax.
- L5: Memory allocation during dense ranking of values.

CORE INTERVIEW QUESTION

"Explain the behavioral difference between `ROW_NUMBER()`, `RANK()`, and `DENSE_RANK()` when sorting data with duplicate values."

DENSE_RANK

CONTIGUOUS RANK

DUPLICATE SORTING

TIE BEHAVIORS

MODEL ANSWER

"Given three rows with identical sorting values:

`ROW_NUMBER()` outputs unique sequential values: 1, 2, 3.

`RANK()` assigns identical ranks and leaves gaps: 1, 1, 3.

`DENSE_RANK()` assigns identical ranks without gaps: 1, 1, 2."

HOW TO EXPLAIN IN INTERVIEW

"**Set-Based Logic:** SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

ADVANCED SQL

Yield: ★★★★★

22 – LEAD()

Accessing forward row values and offset offsets.

DEFINITION & KEY CONCEPTS

The `LEAD()` window function allows queries to access data from subsequent rows without performing expensive self-joins. You can specify a forward offset distance (default is 1) and define default fallback values for boundary nulls.

`LEAD(salary, 1, 0)` Flow:

Row 1 (\$1000) → Lead Value = \$2000 (Row 2)
Row 2 (\$2000) → Lead Value = \$0 (Fallback)

WHERE USED IN INDUSTRY

Industrial IoT sensors compare consecutive temperature telemetry points, checking forward rows to flag sudden sensor overheating.

INTERVIEW DEPTH

- L1:** Accessing subsequent records without self-joins.
- L2:** Passable arguments (offset and default fallbacks).
- L3:** Null prevention on partition boundary limits.
- L4:** Execution tree for forward row offset reads.
- L5:** Lead lookups over time-series logs.

CORE INTERVIEW QUESTION

"What does the third argument in the `LEAD(col, offset, default)` function do, and why is it useful?"

FALLBACK VALUE

OFFSET DISTANCE

NULL PREVENTION

WINDOW BOUNDARY

MODEL ANSWER

"The third argument defines a fallback value returned when the window calculation reaches the boundary (i.e. when the forward offset points beyond the end of the partition). This prevents database outputs from returning NULLs, simplifying delta calculations."

HOW TO EXPLAIN IN INTERVIEW

"Set-Based Logic: SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

ADVANCED SQL

Yield: ★★★★★

23 – LAG()

Accessing backward row values and time-series deltas.

DEFINITION & KEY CONCEPTS

The `LAG()` window function enables access to preceding row values relative to the current row. This is optimal for time-series analysis (e.g. comparing month-over-month sales or daily server metric fluctuations) at $O(N)$ cost.

LAG(sales, 1) Flow:

Jan (\$5k) → Lag Value = NULL
Feb (\$8k) → Lag Value = \$5k (Jan)



WHERE USED IN INDUSTRY

SaaS subscription logs compute month-over-month recurring revenue deltas by comparing current logs with preceding month values.



INTERVIEW DEPTH

- L1:** Accessing preceding records without self-joins.
- L2:** Lag offset steps and boundary default constants.
- L3:** Calculating percentages and deltas over intervals.
- L4:** Time-series lag seek operations on sorted indexes.
- L5:** Comparing lag utility vs self-join complexity.

CORE INTERVIEW QUESTION

"How would you write a query to find the percentage increase in sales month-over-month using the `LAG()` function?"

DELTA CALCULATION

TIME-SERIES ANALYSIS

OFFSET LAG

PRECEDING ROW

MODEL ANSWER

```
"To calculate sales deltas: SELECT month, sales,
LAG(sales) OVER (ORDER BY month) AS prev_sales,
((sales - LAG(sales) OVER (ORDER BY month)) /
LAG(sales) OVER (ORDER BY month)) * 100 AS
pct_increase FROM monthly_sales."
```



HOW TO EXPLAIN IN INTERVIEW

"Set-Based Logic: SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

⚠ COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

🔍 TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

🔴 INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.



MEMORY TRICK

Alance sunlinith mar handividh and neaunnt

INDEXING & PERFORMANCE

Yield: ★★★★★

24 – Database Indexing

B-Tree node storage and index seeks.

DEFINITION & KEY CONCEPTS

An Index is an auxiliary disk structure that speeds up database record retrieval. Most relational indexes utilize **B-Trees** (specifically B+ Trees), maintaining sorted values in a tree hierarchy (Root → Internal Nodes → Leaf Nodes) to reduce lookup complexity to $O(\log N)$.

Index Search Types:

- **Index Seek:** Traverse the B-Tree directly to locate the match (Fast).
- **Index Scan:** Traverse the entire sorted index leaf list (Moderate).
- **Table Scan:** Scan the unordered table on disk (Slow).

WHERE USED IN INDUSTRY

Relational storage backends write indexes to disk structures to query keys in microseconds instead of scanning tables.

INTERVIEW DEPTH

- L1:** Disk storage paths and lookup optimization.
- L2:** B+ Tree node traversal mechanics ($O(\log N)$ seeking).
- L3:** Index scans vs index seeks in execution plans.
- L4:** Write degradation: indexing overhead during writes.
- L5:** Updating index statistics to keep plans cost-accurate.

CORE INTERVIEW QUESTION

"What is the physical difference between an Index Seek and an Index Scan in database execution plan outputs?"

B+ TREE

LEAF NODE

INDEX SEEK

INDEX SCAN

MODEL ANSWER

"An **Index Seek** traverses the B-Tree from the root node to locate specific matching record leaf keys directly. An **Index Scan** reads all keys sequentially along the index leaf node linked-list, which happens when filter predicates are not matching key orders."

HOW TO EXPLAIN IN INTERVIEW

Index Traversals: B+ Trees store keys in sorted order. Index seeks traverse paths in $O(\log N)$ operations. Scans occur when predicates are non-SARGable, requiring sequential scans of leaf nodes."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

INDEXING & PERFORMANCE

Yield: ★★★★★

25 – Clustered Index

Physical sorting and primary key tables.

DEFINITION & KEY CONCEPTS

A Clustered Index dictates the physical sorting order of table rows on disk. In a clustered index, the leaf nodes of the B+ Tree contain the **actual data pages** of the table. Because physical data can only be sorted one way, a table can only have **one** clustered index.

Root Node → Branch Nodes
→ Leaf Nodes (Contain Actual Data Rows 101, 102, 103)



WHERE USED IN INDUSTRY

Relational databases physically sort tables by their primary key, resolving record lookups at the root index nodes.



INTERVIEW DEPTH

- L1: Physical disk page sorting by index key.
- L2: Leaf node contents: Actual data pages (Clustered).
- L3: Single clustered index limit per database relation.
- L4: Auto-increment PK design vs UUID index fragmentation.
- L5: Page split events during non-sequential index insertions.

CORE INTERVIEW QUESTION

"Does creating a Primary Key constraint automatically create a clustered index, and why can you only have one clustered index?"

PHYSICAL SORTING

PRIMARY KEY

LEAF DATA PAGE

INDEX DEFAULT

MODEL ANSWER

"Creating a Primary Key automatically builds a clustered index by default (can be overridden). Because table data pages can only be sorted physically in one physical order (e.g. sorted by ID or sorted by Date), a table can only support one clustered index."



HOW TO EXPLAIN IN INTERVIEW

"Set-Based Logic: SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

INDEXING & PERFORMANCE

Yield: ★★★★★

26 – Non-Clustered Index

Logical pointers and key lookup overhead.

DEFINITION & KEY CONCEPTS

A Non-Clustered Index is a logical structure separated from data pages. Leaf nodes of non-clustered B+ Trees do not contain data rows; they store index key values and ****pointers**** (Row IDs or Clustered Index Keys) pointing to the physical data rows.

```
Leaf Node [Key = "Active"]
  → Pointer → Row ID 412 (Heap Table)
  → Pointer → PK Key 102 (Clustered Table)
```

WHERE USED IN INDUSTRY

High-throughput catalogs map secondary indexes (like product SKU) back to primary key records to query items fast.

INTERVIEW DEPTH

- L1:** Logical index structures separated from data.
- L2:** Leaf node contents: Key values and data pointers.
- L3:** Key Lookup (or RID Lookup) overheads in queries.
- L4:** Covering indexes: using `INCLUDE` to avoid lookups.
- L5:** Updating pointer tables during clustered index page splits.

CORE INTERVIEW QUESTION

"What is a Key Lookup (or RID Lookup) operation in execution plans, and how can it be avoided?"

KEY LOOKUP

ROW IDENTIFIER

COVERING INDEX

INCLUDE CLAUSE

MODEL ANSWER

"A Key Lookup occurs when a query uses a non-clustered index, but requests columns not stored in the index. The database must traverse the clustered index to fetch those columns. It can be avoided by creating a ****Covering Index**** that contains those columns using the `INCLUDE` clause."

HOW TO EXPLAIN IN INTERVIEW

"Set-Based Logic: SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handvidith and neaunnt

INDEXING & PERFORMANCE

Yield: ★★★★★

27 – Query Optimization

SARGable filtering and optimizing execution paths.

DEFINITION & KEY CONCEPTS

Query Optimization is the process of improving SQL query performance. The optimizer evaluates potential execution plans (Join orderings, index usage, access paths) and selects the lowest-cost path. Avoiding table scans and sargable filters are key optimizations.

Pattern	Status	Alternative
col LIKE '%abc'	Bad Scan	Full Text Search
col LIKE 'abc%'	Good Seek	Index Seek (prefix)
OR checks	Bad Scan	UNION ALL

WHERE USED IN INDUSTRY

DBAs inspect slow query logs, rewriting wildcard filters to ensure execution plans use indexed ranges.

INTERVIEW DEPTH

- L1: Identifying query execution path bottlenecks.
- L2: Non-SARGable wildcard prefixes (LIKE '%value').
- L3: Rewrite paths: OR statements to UNION ALL splits.
- L4: Implicit conversions: string comparing to integers.
- L5: Optimizer cost modeling under outdated statistics.

CORE INTERVIEW QUESTION

"Why does a query filtering with **LIKE** '%value%' fail to use an index, and what alternatives exist?"

WILDCARD PREFIX

LEFT-TO-RIGHT SORT

FULL TABLE SCAN

FULL-TEXT INDEX

MODEL ANSWER

"Indexes are sorted left-to-right. A wildcard prefix (%value) prevents the engine from utilizing sorting sequence, forcing a full table scan. The alternatives are removing the leading wildcard (using prefix matches value%) or using Full-Text Indexes."

HOW TO EXPLAIN IN INTERVIEW

"**Set-Based Logic:** SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

INDEXING & PERFORMANCE

Yield: ★★★★★

28 – Execution Plan Basics

Logical vs physical plan parsing and database costs.

DEFINITION & KEY CONCEPTS

An Execution Plan is the database engine's step-by-step roadmap for executing a query. In SQL, you review the plan using the `EXPLAIN` statement, which lists operators (Nested Loops, Hash Joins, Scans, Seeks) and estimated card costs.

Logical Execution Phases:

Parser (Syntax Check) → Optimizer (Cost Estimate) → Execution Engine (Physical CPU/IO Read)

WHERE USED IN INDUSTRY

Performance engineers review `EXPLAIN` output operators to identify hash join bottlenecks on non-indexed columns.

INTERVIEW DEPTH

- L1: Extracting execution roadmaps using `EXPLAIN`.
- L2: Cost estimations based on CPU and disk page access.
- L3: Estimated vs Actual rows (`EXPLAIN ANALYZE` comparison).
- L4: Logical parser, compiler, and physical execution engine.
- L5: Graph analysis of physical database operations.

CORE INTERVIEW QUESTION

"How do you extract the execution plan of a SQL query, and what is estimated cost?"

EXPLAIN STATEMENT

COST MODEL

I/O UNITS

CARDINALITY ESTIMATE

MODEL ANSWER

"You extract execution plans using `EXPLAIN SELECT ...` (or `EXPLAIN ANALYZE` to run and trace execution). The 'cost' represents estimated CPU cycles and disk page I/O reads calculated using table statistics. Planners compare costs to select the cheapest plan."

HOW TO EXPLAIN IN INTERVIEW

"**Set-Based Logic:** SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

TRANSACTIONS

Yield: ★★★★★

29 – ACID Properties

Transaction guarantees and reliability foundations.

DEFINITION & KEY CONCEPTS

ACID represents the core guarantees of transactions:
Atomicity (All-or-Nothing execution), **Consistency** (Transition from valid state to valid state), **Isolation** (Concurrency control segregation), and **Durability** (Committed data survives system crashes).

ACID Mechanisms:

- **Atomicity/Durability:** Managed by WAL and Undo/Redo Logs.
- **Isolation:** Enforced using Locks or Multi-Version Concurrency (MVCC).

WHERE USED IN INDUSTRY

Banking ledgers write deposits to disk log buffers before modifying schemas, guaranteeing transactions survive host crashes.

INTERVIEW DEPTH

- L1:** ACID guarantees for transactional stability.
- L2:** Write-Ahead Logging (WAL) for durability execution.
- L3:** Undo/Redo segments and logging page checkpoints.
- L4:** Consistency checks using DB constraints vs app logic.
- L5:** Doublewrite buffer safety to prevent page splits.

CORE INTERVIEW QUESTION

"How does the database guarantee Atomicity and Durability during sudden hardware power failures?"

WRITE-AHEAD LOGGING

REDO LOG

UNDO SEGMENT

CRASH RECOVERY

MODEL ANSWER

"The engine guarantees Atomicity and Durability using Write-Ahead Logging (WAL) and Undo/Redo logs. Prior to table modification, transaction steps are written to disk logs. On reboot, the engine scans the logs: committed steps are re-applied (Redo) and uncommitted steps are reversed (Undo)."

ACID Commit Checkpoint Flow:

Transaction SQL → Log Buffer → Flush to WAL Disk Log (Durability)
→ Table Space Pages Modified.

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

TRANSACTIONS

Yield: ★★★★★

30 – Transactions

Isolation levels and concurrency anomalies.

DEFINITION & KEY CONCEPTS

A Transaction is a single logical unit of database work. Managing transaction isolation levels controls concurrency anomalies (Dirty Reads, Non-Repeatable Reads, Phantom Reads) at the cost of processing speed.

Isolation Level	Dirty Read	Non-Rep Read	Phantom Read
Read Uncomm.	Yes	Yes	Yes
Read Comm.	No	Yes	Yes
Repeatable Read	No	No	Yes (MySQL No)
Serializable	No	No	No

WHERE USED IN INDUSTRY

Flight reservation engines set transaction isolation levels to prevent double-booking seats during concurrent checkout requests.

INTERVIEW DEPTH

- L1: Atomic execution blocks: BEGIN, COMMIT, ROLLBACK.
- L2: Dirty Reads, Non-Repeatable Reads, and Phantom Reads.
- L3: Database Isolation Levels and concurrency anomalies.
- L4: Multi-Version Concurrency Control (MVCC) snapshot reads.
- L5: Range locks and gap locking to prevent phantom rows.

CORE INTERVIEW QUESTION

"What is the difference between a Non-Repeatable Read and a Phantom Read, and how are they prevented?"

DIRTY READ

REPEATABLE READ

RANGE LOCK

MVCC SNAPSHOTS

MODEL ANSWER

"A **Non-Repeatable Read** occurs when Transaction A reads a row, Transaction B updates it, and Transaction A reads the same row again to find updated values. A **Phantom Read** occurs when Transaction A reads a range of rows, Transaction B inserts new rows in that range, and Transaction A re-reads the range to find new rows. They are prevented using MVCC or Range Locks."

HOW TO EXPLAIN IN INTERVIEW

Set-Based Logic: SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividh and neaunnt

TRANSACTIONS

Yield: ★★★★★

31 – Locks & Concurrency

Shared vs exclusive locks and deadlocks.

DEFINITION & KEY CONCEPTS

Locks manage concurrent access to database data. The core lock types are ****Shared (S) Locks**** (read locks) and ****Exclusive (X) Locks**** (write locks). Conflict matrices dictate that multiple sessions can read, but only one can write.

Lock Matrix	Shared (S)	Exclusive (X)
Shared (S)	Granted	Blocked
Exclusive (X)	Blocked	Blocked

WHERE USED IN INDUSTRY

Inventory catalogs execute write locks on stock keys during flash sales to prevent purchasing items that have sold out.

INTERVIEW DEPTH

- L1:** Shared (S) locks vs Exclusive (X) locks.
- L2:** Deadlock cycles and dependency graph resolution.
- L3:** Lock escalation (Row → Page → Table lock swaps).
- L4:** Two-Phase Locking (2PL) protocols and serializability.
- L5:** Shared vs Exclusive lock collision resolution.

CORE INTERVIEW QUESTION

"What is a Deadlock, and how does the database engine detect and resolve it?"

DEADLOCK CYCLE

LOCK WAIT TIMEOUT

TRANSACTION ROLLBACK

DEPENDENCY GRAPH

MODEL ANSWER

"A Deadlock occurs when Transaction A holds Lock 1 and requests Lock 2, while Transaction B holds Lock 2 and requests Lock 1. The engine detects this cycle using dependency graphs. It resolves it by terminating one transaction (the victim), rolling back its steps, and releasing its locks."

HOW TO EXPLAIN IN INTERVIEW

"Set-Based Logic: SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

INTERVIEW SQL

Yield: ★★★★★

32 – Nth Highest Salary

Correlated subqueries vs window functions.

DEFINITION & KEY CONCEPTS

Finding the Nth highest salary is a classic interview query. It checks the candidate's understanding of subqueries, window functions, and tie-handling. The two standard methods are correlated subqueries and `DENSE_RANK()` functions.

Method 1: Correlated Subquery

```
SELECT DISTINCT salary FROM Employee E1
WHERE N-1 = (
    SELECT COUNT(DISTINCT salary)
    FROM Employee E2 WHERE E2.salary > E1.salary
)
```

WHERE USED IN INDUSTRY

HR compliance platforms calculate gender-pay metrics, querying salary values at specific partition index points.

INTERVIEW DEPTH

- L1: Subqueries to locate top N salary metrics.
- L2: `DENSE_RANK()` OVER (ORDER BY salary DESC) queries.
- L3: Rank tie differences (RANK vs `DENSE_RANK` gaps).
- L4: Correlated subquery complexity ($O(N^2)$ evaluation).
- L5: Index-based seeks on ordered salary fields.

CORE INTERVIEW QUESTION

"Write a query to find the Nth highest salary using `DENSE_RANK()`, and explain why `DENSE_RANK()` is preferred over `RANK()`."

DENSE_RANK

WINDOW FILTERING

DUPLICATE SALARY

RANK TIES

MODEL ANSWER

"Query:

```
WITH RankedSalary AS (SELECT salary,
DENSE_RANK() OVER (ORDER BY salary DESC) as rk
FROM Employee) SELECT salary FROM RankedSalary
WHERE rk = N LIMIT 1;
DENSE_RANK() is preferred because if salaries are tied (e.g.
$10k, $10k, $8k), DENSE_RANK() assigns (1, 1, 2) finding $8k as
the 2nd highest, whereas RANK() assigns (1, 1, 3), skipping
rank 2."
```

HOW TO EXPLAIN IN INTERVIEW

"Set-Based Logic: SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

INTERVIEW SQL

Yield: ★★★★★

33 – Duplicate Records

Identifying and cleaning database records.

DEFINITION & KEY CONCEPTS

Duplicate records are matching values in key columns. Identifying duplicates uses `GROUP BY` and `HAVING COUNT(\*) > 1`. Removing duplicates requires deleting duplicate rows while keeping the unique primary key record.

Delete Duplicate Rows:

```
DELETE FROM Users WHERE id NOT IN (  
    SELECT MIN(id) FROM Users  
    GROUP BY email  
)
```

WHERE USED IN INDUSTRY

CRM platforms clean contact lists, running row-number deduplication scripts to delete duplicate profile registrations.

INTERVIEW DEPTH

- L1: Finding duplicates with GROUP BY and HAVING COUNT.
- L2: ROW_NUMBER() window partitioning for record isolation.
- L3: Bulk deletion of duplicate rows keeping minimum ID.
- L4: Unique indexes to prevent duplicate insert attempts.
- L5: Locking concerns during bulk duplicate cleaning operations.

CORE INTERVIEW QUESTION

"How would you delete all duplicate records based on email, keeping only the record with the lowest unique ID?"

GROUP BY

SUBQUERY EXCLUSION

ROW_NUMBER() DELETION

DEDUPLICATION

MODEL ANSWER

"Using CTE:

```
WITH CTE AS (SELECT id, ROW_NUMBER() OVER  
    (PARTITION BY email ORDER BY id) as rn FROM  
    Users) DELETE FROM Users WHERE id IN (SELECT id  
    FROM CTE WHERE rn > 1);
```

This partitions records by email and numbers them. Any row with a row number greater than 1 represents a duplicate and is deleted."

HOW TO EXPLAIN IN INTERVIEW

"Set-Based Logic: SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming SELECT * has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand * to all columns.

INTERVIEWER TRAP

Claiming SELECT * has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand * to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

INTERVIEW SQL

Yield: ★★★★★

34 – Consecutive Numbers

Sequence isolation and row number differences.

DEFINITION & KEY CONCEPTS

Finding consecutive repeating numbers is an advanced query pattern. The standard interview problem asks to identify numbers that appear at least three times consecutively. This can be resolved using multiple self-joins or `LEAD`/`LAG` window functions.

Id	Num	Prev (LAG 1)	Prev2 (LAG 2)
1	10	NULL	NULL
2	10	10	NULL
3	10	10	10 (Consecutive!)

WHERE USED IN INDUSTRY

Security systems analyze user logs, querying for accounts with three consecutive failed login attempts to trigger security lockouts.

INTERVIEW DEPTH

- L1:** Identifying repeating values over sequences.
- L2:** LEAD() and LAG() offset comparisons (offset 1 and 2).
- L3:** Row number difference techniques (Gaps and Islands).
- L4:** Handling gaps in ID columns during self-joins.
- L5:** Optimizing sequential range scans on logs.

CORE INTERVIEW QUESTION

"Write a query to find all numbers that appear at least three times consecutively."

CONSECUTIVE SEQUENCES

LAG WINDOW

GAPS AND ISLANDS

CONDITIONAL JOINS

MODEL ANSWER

"Query using LAG:

```
WITH SeqCheck AS (SELECT Num, LAG(Num, 1) OVER
  (ORDER BY id) as prev1, LAG(Num, 2) OVER (ORDER
  BY id) as prev2 FROM Logs) SELECT DISTINCT Num
FROM SeqCheck WHERE Num = prev1 AND Num =
prev2;
```

This fetches the preceding two rows for each row and verifies if they have the same value."

HOW TO EXPLAIN IN INTERVIEW

"Set-Based Logic: SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming SELECT * has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand * to all columns.

INTERVIEWER TRAP

Claiming SELECT * has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand * to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

INTERVIEW SQL

Yield: ★★★★★

35 – Department Top Earners

Group partitioning and top rank extraction.

DEFINITION & KEY CONCEPTS

Finding the department's highest salary requires partitioning rows. An inner subquery calculates ranks per department using the `DENSE\_RANK() OVER (PARTITION BY dept ORDER BY salary DESC)` window function, and the outer query filters rank 1.

Partition Windowing:

- Dept A: Emp 1 (\$50k) → Rank 1
- Dept A: Emp 2 (\$45k) → Rank 2
- Dept B: Emp 3 (\$60k) → Rank 1

WHERE USED IN INDUSTRY

Payroll dashboards calculate department top earners, reporting employees who receive the maximum salary in each team.

INTERVIEW DEPTH

- L1:** Group partitioning window calculations.
- L2:** `DENSE_RANK() PARTITION BY department ORDER BY salary`.
- L3:** Handling duplicate maximum value occurrences.
- L4:** Index strategies for partition sorting.
- L5:** CTE vs nested subquery performance paths.

CORE INTERVIEW QUESTION

"Write a query to find employees who have the highest salary in each of their departments."

PARTITION BY

DENSE_RANK

PARTITION BOUNDARY

FILTER SUBQUERY

MODEL ANSWER

"Query:

```
WITH RankedSalary AS (SELECT name, salary,
dept_id, DENSE_RANK() OVER (PARTITION BY
dept_id ORDER BY salary DESC) as rk FROM
Employee) SELECT name, salary, dept_id FROM
RankedSalary WHERE rk = 1;
```

This partitions records by department, ranks salaries descending, and extracts all rows with rank 1 (handling tied top salaries)."

HOW TO EXPLAIN IN INTERVIEW

"Set-Based Logic: SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

INTERVIEW SQL

Yield: ★★★★★

36 – Running Total

Cumulative aggregations and window frame clauses.

DEFINITION & KEY CONCEPTS

Calculating running totals or cumulative sums computes progress over time. In SQL, this is achieved using window aggregation: `SUM(val) OVER (ORDER BY date)`. Understanding window frame clauses is critical for performance.

Date	Amount	Running Total
Jan 1	100	100
Jan 2	150	250 (100+150)
Jan 3	200	450 (250+200)

WHERE USED IN INDUSTRY

SaaS sales dashboards track running totals to display year-to-date cumulative revenue figures over monthly intervals.

INTERVIEW DEPTH

- L1:** Running aggregates over sorting indexes.
- L2:** Window frame clauses: `ROWS BETWEEN UNBOUNDED` and `CURRENT`.
- L3:** $O(N)$ execution plans using aggregate window buffers.
- L4:** Omitting `ORDER BY`: partition-level summary defaults.
- L5:** Range-based vs Row-based window frame evaluations.

CORE INTERVIEW QUESTION

"Explain what the frame clause `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` does and why it is useful."

WINDOW FRAME

RUNNING TOTAL

UNBOUNDED

CUMULATIVE SUM

MODEL ANSWER

"The frame clause defines the boundaries of rows included in the current aggregate. `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` instructs the engine to sum values from the first row of the partition up to the current row, calculating a running total."

HOW TO EXPLAIN IN INTERVIEW

"**Set-Based Logic:** SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

INTERVIEW SQL

Yield: ★★★★★

37 – Top N Per Group

Filtering window rankings in subqueries.

DEFINITION & KEY CONCEPTS

Extracting the top N records per group (e.g. top 3 customers per region) builds upon partitioning. It is resolved by assigning ranks using `ROW\_NUMBER()` or `DENSE\_RANK()`, and filtering for rank $\leq N$ in the outer query block.

Top 2 Per Group Flow:

Region A: Cust 1 (Rank 1) → Retained
Region A: Cust 2 (Rank 2) → Retained
Region A: Cust 3 (Rank 3) → Discarded

WHERE USED IN INDUSTRY

E-commerce frontends display the top 3 highest-rated products in each product category using window partition rankings.

INTERVIEW DEPTH

- L1: Partition rankings and top list extractions.
- L2: `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)` logic.
- L3: Filtering window values inside outer query wrappers.
- L4: `DENSE_RANK` tie resolution in group limits.
- L5: Performance analysis of partition seeks.

CORE INTERVIEW QUESTION

"Write a query to extract the top 3 highest-spending customers for each country."

TOP N FILTER

ROW_NUMBER

CTE SELECTION

GROUP RANKINGS

MODEL ANSWER

"Query:

```
WITH RankedCustomers AS (SELECT customer_id,
country, spending, ROW_NUMBER() OVER (PARTITION
BY country ORDER BY spending DESC) as rn FROM
CustomerSpend) SELECT customer_id, country,
spending FROM RankedCustomers WHERE rn <= 3;
This numbers rows per country descending by spending,
and returns rows numbered 1, 2, or 3."
```

HOW TO EXPLAIN IN INTERVIEW

"Set-Based Logic: SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

INTERVIEW SQL

Yield: ★★★★★

38 – Customer Retention

Cohort analysis and active user retention.

DEFINITION & KEY CONCEPTS

Customer retention measures repeat user logins. A standard query tracks Month-over-Month (MoM) active user retention. This is solved by self-joining user activity logs on user identifiers with matching month offsets.

Active Users Month 1 (Join on UserID)
→ Active Users Month 2

- Formula: $\text{Retained} / \text{Total Month 1} \times 100$

WHERE USED IN INDUSTRY

Growth metrics systems calculate user retention percentages by tracking user login cohorts month-over-month.

INTERVIEW DEPTH

- L1: Cohort metrics and active user mappings.
- L2: Self-joins on user IDs with matching month offsets.
- L3: LEFT JOIN preservation to compute retention percentages.
- L4: Cohort boundaries and active client classifications.
- L5: Optimizing massive user login self-joins.

CORE INTERVIEW QUESTION

"Write a query to find the percentage of active users in Month 1 who logged in again in Month 2."

COHORT SELF-JOIN ACTIVE USER

MONTHLY ACTIVE USERS

MODEL ANSWER

"Query:

```
SELECT COUNT(DISTINCT M2.user_id) * 100.0 /  
COUNT(DISTINCT M1.user_id) AS retention_rate  
FROM Logins M1 LEFT JOIN Logins M2 ON  
M1.user_id = M2.user_id AND M2.month = M1.month  
+ 1 WHERE M1.month = 1;
```

This left-joins Month 1 logins to Month 2 logins on user ID, calculating the percentage that returned."

HOW TO EXPLAIN IN INTERVIEW

"Set-Based Logic: SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

INTERVIEWER TRAP

Claiming `SELECT *` has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand `*` to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

INTERVIEW SQL

Yield: ★★★★★

39 – Ranking Queries

Complex custom ranking constraints and ties.

DEFINITION & KEY CONCEPTS

Complex ranking problems require sorting data based on secondary conditional columns when ties occur. This is implemented by supplying multiple columns to the window `ORDER BY` clause (e.g. sort by score DESC, then by date ASC).

Compound Order Priority:

1. Score DESC (Primary)
2. Errors ASC (Tie Breaker)
3. Timestamp ASC (Secondary Tie Breaker)

WHERE USED IN INDUSTRY

Trading engines rank transactions by amount descending and time ascending to resolve order execution sequence.

INTERVIEW DEPTH

- L1:** Sorting data with multiple tie-breaking criteria.
- L2:** Compound ORDER BY lists inside OVER() partition blocks.
- L3:** Ascending vs Descending order priority limits.
- L4:** Deterministic rank values using unique identifiers.
- L5:** Index sorting paths for compound ordering keys.

CORE INTERVIEW QUESTION

"Write a query to rank students by score descending. If scores are equal, rank them by exam completion time ascending."

DENSE_RANK

SECONDARY SORT

TIE BREAKING

SORT ORDER

MODEL ANSWER

"Query:

```
SELECT student_id, score, completion_time,  
DENSE_RANK() OVER (ORDER BY score DESC,  
completion_time ASC) as rk FROM ExamResults;  
By specifying multiple columns in ORDER BY, ties in score are  
resolved by completion time before rank assignment."
```

HOW TO EXPLAIN IN INTERVIEW

"Set-Based Logic: SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming SELECT * has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand * to all columns.

INTERVIEWER TRAP

Claiming SELECT * has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand * to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

INTERVIEW SQL

Yield: ★★★★★

40 – Date Queries

DATEDIFF, DATEADD, and date truncation.

DEFINITION & KEY CONCEPTS

Date metrics require handling temporal functions. The most tested operations are DATEDIFF (calculating days between dates), DATEADD (moving dates forward), and DATE_TRUNC (rounding timestamps to boundaries).

```
DATE_TRUNC('month', '2026-05-30')  
→ Output: '2026-05-01 00:00:00'
```

- Useful for group aggregation by month.

WHERE USED IN INDUSTRY

Compliance systems query user creation dates, truncating timestamps to aggregate logs by month for tax audits.

INTERVIEW DEPTH

- L1: Temporal data manipulation and parsing paths.
- L2: DATEDIFF, DATEADD, and DATE_TRUNC operations.
- L3: Collapsing timestamps to monthly intervals for grouping.
- L4: Timezone offsets and date math complications.
- L5: Range queries on index date columns.

CORE INTERVIEW QUESTION

"Write a query to count the number of users registered per month, truncating creation timestamps."

DATE_TRUNC

TEMPORAL AGGREGATION

DATE MATH

TIME PERIOD GROUPING

MODEL ANSWER

"Query:

```
SELECT DATE_TRUNC('month', created_at) AS  
reg_month, COUNT(*) AS count FROM Users GROUP  
BY reg_month ORDER BY reg_month;
```

Using DATE_TRUNC collapses all daily timestamps to their respective monthly boundaries for clean grouping."

HOW TO EXPLAIN IN INTERVIEW

"**Set-Based Logic:** SQL is a declarative language. Explain that you define 'what' data to fetch, leaving the database engine's optimizer to determine the most efficient physical path (joins, seeks, and indexes)."

COMMON MISTAKE

Believing databases store data sequentially by default. Relational data is an unordered bag unless sorted or indexed.

TOP FOLLOW-UP

Claiming SELECT * has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand * to all columns.

INTERVIEWER TRAP

Claiming SELECT * has the exact same performance as explicit projection. It requires an extra dictionary metadata lookup to expand * to all columns.

MEMORY TRICK

Alance sunlinith mar handividth and neaunnt

SQL Cram & Execution Guide

Logical Processing Steps & Transaction Anomalies

⚙️ LOGICAL QUERY PROCESSING ORDER

1. **FROM:** Identifies source tables and builds Cartesian product grid.
2. **ON:** Filters join rows based on join conditional keys.
3. **JOIN:** Joins target relations (INNER, LEFT, RIGHT).
4. **WHERE:** Filters individual row records (pre-aggregation).
5. **GROUP BY:** Groups rows by specified unique columns.
6. **HAVING:** Filters aggregated group results (post-aggregation).
7. **SELECT:** Projects column selections and window calculations.
8. **DISTINCT:** Deduplicates identical rows.
9. **ORDER BY:** Sorts rows logically (filesort vs index scan).
10. **LIMIT / OFFSET:** Constrains output row limits.

⚡ ACID CONCURRENCY

Dirty Read	Read uncommitted transactions.
Non-Rep	Values modify mid-transaction.
Phantom	Rows inserted in selected ranges.
S-Lock	Shared read locks (compatible).
X-Lock	Exclusive write locks (blocking).

💡 SARGABLE PREDICATE OPTIMIZATIONS

- **Non-SARGable:** `WHERE DATE_ADD(created, INTERVAL 7 DAY) > NOW()` (Forces table scan)
- **SARGable Rewrite:** `WHERE created > DATE_SUB(NOW(), INTERVAL 7 DAY)` (Leverages index seek)
- **Wildcard Scan:** `WHERE name LIKE '%xyz%'` (Index Scan)
- **Prefix Seek:** `WHERE name LIKE 'xyz%'` (Index Range Seek)

Most Asked SQL Comparisons

Key Architectural Differences Summarized for Placement Interviews

Topic	Option A	Option B	Key Difference
Filter	WHERE: Filters rows prior to grouping.	HAVING: Filters aggregated groups.	WHERE cannot filter aggregate functions (e.g. SUM); HAVING operates post-aggregation.
Deduplicate	DISTINCT: Evaluates row values for deduplication.	GROUP BY: Groups rows by unique key sets.	DISTINCT is for data projection uniqueness; GROUP BY splits rows for aggregate operations.
Index	Clustered Index: Data physically sorted on disk.	Non-Clustered: Separated logical lookup pointers.	One clustered index per table (keys contain actual data); multiple non-clustered allowed.
Remove	DELETE: DML, filters rows, writes WAL logs.	TRUNCATE: DDL, drops and reallocates pages.	DELETE supports WHERE and triggers; TRUNCATE is faster and releases data pages directly.
Rank	RANK(): Assigns identical rank with gaps.	DENSE_RANK(): Continuous ranking values.	RANK() skips ranking steps on ties (1, 1, 3); DENSE_RANK() leaves no gaps (1, 1, 2).
Subquery	Subquery: Executed once independently.	Correlated: References outer query columns.	Subqueries evaluate once; correlated subqueries loop row-by-row for the outer set.

SQL Top Expected Interview Questions

Placement Q&As · Created by Pranav Gawai (Part 1)

Q1: Explain the functional differences between DELETE, TRUNCATE, and DROP commands in SQL.

DELETE is a Data Manipulation Language (DML) command that deletes specific rows using a WHERE clause. It writes individual row deletes to the Write-Ahead Log (WAL), triggers database triggers, and retains index space. TRUNCATE is a Data Definition Language (DDL) command that drops the table pages and reallocates storage pages directly. It writes minimal page-deallocation steps to the logs, bypasses delete triggers, and resets auto-increment primary keys. DROP is also a DDL command that removes the entire table structure, indexes, permissions, and definition from the database catalog entirely.

Keywords: DML vs DDL · Page Reallocation · Triggers Bypass · WAL Logging Log

Senior Depth: DELETE acquires Row-Exclusive locks. TRUNCATE locks the entire table page allocation units (Exclusive schema lock).

Q2: Compare ROW_NUMBER(), RANK(), and DENSE_RANK() window functions. Show their outputs for tied scores [100, 100, 90].

ROW_NUMBER() assigns a unique, sequential row index. For ties, it resolves numbers arbitrarily: [1, 2, 3]. RANK() assigns identical rankings to tied values, but leaves gaps in sequential values: [1, 1, 3] (rank 2 is skipped because 2 rows share rank 1). DENSE_RANK() assigns identical rankings to tied values, but keeps the sequence contiguous: [1, 1, 2]. All three require the OVER() clause specifying ordering columns.

Keywords: Contiguous Rank · Sequence Gaps · Arbitrary Sequence · Tied Scores

Senior Depth: Explain sorting costs in execution plan sorting steps (Sort operator overhead).

SQL Top Expected Interview Questions

Placement Q&As · Created by Pranav Gawai (Part 2)

Q3: Write a SQL query to find the Nth highest salary in the Employee table using both CTE and subqueries.

Subquery Method:

```
SELECT DISTINCT salary FROM Employee E1 WHERE N-1 = (SELECT COUNT(DISTINCT salary) FROM Employee E2 WHERE E2.salary > E1.salary);
```

DENSE_RANK CTE Method:

```
WITH RankedSalary AS (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) as rk FROM Employee) SELECT salary FROM RankedSalary WHERE rk = N LIMIT 1;
```

Keywords: DENSE_RANK() · Correlated Check · CTE Filter · Offset Seek

Senior Depth: The CTE Dense Rank method is generally faster due to set-based sorting optimizations instead of the nested $O(N^2)$ loops of correlated queries.

Q4: How do you identify and delete duplicate user rows based on email, keeping only the record with the lowest primary key ID?

To locate duplicates: `SELECT email, COUNT(*) FROM Users GROUP BY email HAVING COUNT(*) > 1;` To delete duplicates: `WITH DuplicateRows AS (SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn FROM Users) DELETE FROM Users WHERE id IN (SELECT id FROM DuplicateRows WHERE rn > 1);` This numbers duplicate emails sequentially by ID and deletes any duplicate row indexed higher than 1.

Keywords: ROW_NUMBER() Deletion · PARTITION BY · HAVING COUNT > 1 · Subquery filter

Senior Depth: In massive tables, batch duplicate deletions in intervals (e.g., LIMIT 5000) to prevent transaction log expansion.

SQL Top Expected Interview Questions

Placement Q&As · Created by Pranav Gawai (Part 3)

Q5: What is a covering index, and how does the INCLUDE clause improve query performance?

A covering index is a non-clustered index that contains all columns requested by a query (both filters and projections). When a index covers a query, the database resolves the request using the index leaf nodes directly. The INCLUDE clause appends non-key columns to the index leaf nodes without sorting them, avoiding the physical clustered index traversal known as Key/RID lookup.

Keywords: Key Lookup · RID Lookup · INCLUDE Key · Leaf Node Content

Senior Depth: Adding columns to the INCLUDE clause avoids index sorting overhead because columns are stored raw at the leaf node level.

Q6: Explain transaction isolation levels and how they prevent dirty reads, non-repeatable reads, and phantom reads.

Read Uncommitted permits dirty reads (reading uncommitted write buffers). Read Committed prevents dirty reads by reading only committed data. Repeatable Read prevents non-repeatable reads (row values modified mid-transaction) using read locks or snapshot views. Serializable prevents phantom reads (new rows inserted in selected ranges) by using range/gap locking. Each level trades concurrency for safety.

Keywords: MVCC Snapshots · Gap Locking · Read Locks · Anomaly Isolation

Senior Depth: Engines like PostgreSQL/MySQL implement Repeatable Read using MVCC, which handles phantom reads without using lock ranges.

SQL Top Expected Interview Questions

Placement Q&As · Created by Pranav Gawai (Part 4)

Q7: Write a query to find all users who logged in for 3 consecutive days, explaining the gap-and-island technique.

Using row number difference:

```
WITH GroupedLogins AS (SELECT user_id, login_date, login_date - ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) * INTERVAL '1 day' AS grp FROM Logins) SELECT DISTINCT user_id FROM GroupedLogins GROUP BY user_id, grp HAVING COUNT(*) >= 3;
```

Subtracting the sequential row index from dates yields identical anchor dates for consecutive login series.

Keywords: Gaps and Islands · Row Number Offset · Date Interval Math · Consecutive Days

Senior Depth: This partitions dates into groups without procedural loops. The CTE evaluates in $O(N \log N)$ sorting time.

Q8: How do you optimize a query with the filter WHERE LOWER(username) = 'john' and why does it run slow?

Wrapping columns in functions like `LOWER()` makes predicates non-SARGable, forcing a full table scan because the engine must compute `LOWER()` for every row. Optimize it by: 1. Making the search case-insensitive at the database schema collation level. 2. Creating a Function-Based Index (or expression index): `CREATE INDEX idx_lower_user ON Users (LOWER(username));`

Keywords: Non-SARGable Predicate · Function-Based Index · Expression Index · Collation Setup

Senior Depth: Function-based indexes compute and store values during write operations, restoring $O(\log N)$ seeking paths.

SQL Top Expected Interview Questions

Placement Q&As · Created by Pranav Gawai (Part 5)

Q9: Write a query to calculate a running total of transactions per account over time.

Running total is computed using window SUM:

```
SELECT account_id, trans_date, amount, SUM(amount) OVER (PARTITION BY account_id ORDER BY trans_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total FROM Transactions;
```

The frame clause instructs the engine to sum values from the start of the account partition up to the current row date.

Keywords: Window Aggregate · Running Total · Frame Bounds · ROWS BETWEEN

Senior Depth: Omitting the ROWS frame clause defaults the frame boundary to RANGE, which is slower because it evaluates duplicate dates together.

Q10: Explain theCAP Theorem and PACELC extension for distributed database designs.

CAP Theorem states that a distributed data system can simultaneously guarantee only two of: Consistency, Availability, and Partition Tolerance. PACELC extends this: If there is a Partition (P), choose Availability (A) or Consistency (C); Else (E), trade Latency (L) or Consistency (C). This models real-world database choices like MongoDB (CP) or Cassandra (AP).

Keywords: CAP Theorem · PACELC Extension · Eventual Consistency · Latency vs Consistency

Senior Depth: Distributed setups like Spanner utilize atomic clocks (TrueTime API) to achieve both consistency and partition tolerance.

SQL Rapid Fire Questions (Part 1)

Fast-Recall Flashcards for Last-Minute Self-Testing

Q: What does SELECT 1 FROM table do?

A: Returns integer 1 for each matching row; used in EXISTS checks.

Q: What is the default sort order of ORDER BY?

A: Ascending (ASC).

Q: How does DISTINCT eliminate duplicates?

A: Uses hash aggregation or sorting algorithms to collapse rows.

Q: What is the danger of high LIMIT offsets?

A: Scans and discards skipped rows, creating database I/O latency.

Q: Can we select columns not in GROUP BY?

A: No, unless they are wrapped in aggregate functions.

Q: Difference between COUNT(*) and COUNT(col)?

A: COUNT(*) counts all rows; COUNT(col) counts only non-null values.

Q: What join produces a Cartesian Product?

A: CROSS JOIN.

Q: Does a LEFT JOIN preserve right-side rows?

A: No, it preserves left-side rows with null padding for right rows.

Q: When is a SELF JOIN useful?

A: Mapping parent-child, supervisor-employee, or recursive structures.

Q: What is a correlated subquery?

A: A subquery that references outer columns, running row-by-row.

Q: What is the benefit of EXISTS over IN?

A: EXISTS short-circuits on the first match, bypassing complete scans.

Q: What does a CTE stand for?

A: Common Table Expression (WITH clause result sets).

Q: Can a CTE be recursive?

A: Yes, referencing itself to traverse tree schemas.

Q: What clause is mandatory for window functions?

A: The OVER clause.

Q: What does LEAD(col, 2) do?

A: Accesses values located 2 rows forward in the window partition.

Q: What is the output of DENSE_RANK() for ties?

A: Assigns identical ranks, incrementing by 1 for the next distinct rank.

SQL Rapid Fire Questions (Part 2)

Fast-Recall Flashcards for Last-Minute Self-Testing

Q: What index structure is used by default?

A: B-Tree (specifically B+ Tree node structures).

Q: How many clustered indexes can a table have?

A: One, because data rows can only be physically sorted one way.

Q: What is a covering index?

A: An index containing all columns selected, filtered, and projected by a query.

Q: What is a SARGable predicate?

A: A search argument that allows the query engine to perform an index seek.

Q: What is an execution plan?

A: The query compiler's step-by-step roadmap for executing SQL.

Q: What does the 'I' in ACID stand for?

A: Isolation (preventing concurrent transaction interference).

Q: What are the 4 database isolation levels?

A: Read Uncommitted, Read Committed, Repeatable Read, Serializable.

Q: What is a dirty read?

A: Reading uncommitted writes from parallel transactions.

Q: What is a phantom read?

A: New rows inserted in a selected range mid-transaction.

Q: What does a COMMIT statement do?

A: Permanently writes transactional modifications to tablespace files.

Q: What is a deadlock?

A: A circular lock block between two transactions.

Q: How does a DB resolve deadlocks?

A: Kills and rolls back the transaction with the lowest cost victim.

Q: What is a Shared Lock (S-Lock)?

A: Read lock allowing concurrent reads but blocking modifications.

Q: What is an Exclusive Lock (X-Lock)?

A: Write lock blocking all parallel access to target rows.

Q: What is the SQL standard date format?

A: YYYY-MM-DD.

Q: What does DATE_TRUNC('month', date) do?

A: Rounds a date timestamp to the first day of that month.



SQL Common Traps (Part 1)

Tactics to Evade Trick Interview Questions with Confident Answers

Trap 1: The Null Arithmetic Trap

Interviewer: "What is the result of executing `SELECT 10 + NULL` in SQL?"

Deflection: State clearly that NULL represents an unknown value. Therefore, any standard arithmetic calculation or comparison containing NULL yields NULL. Always use `COALESCE(col, 0)` to prevent NULL propagation.

Trap 2: The WHERE Aggregate Trap

Interviewer: "Can you filter grouped aggregate values directly inside the `WHERE` clause?"

Deflection: No. The `WHERE` clause executes before grouping takes place. Aggregated values do not exist yet in the database execution pipeline. You must use `HAVING` to filter aggregated metrics.

Trap 3: The Cartesian Explosion Join Trap

Interviewer: "What happens if you run an `INNER JOIN` on a massive table and omit the `ON` condition?"

Deflection: It degenerates into a `CROSS JOIN`, generating the Cartesian product (Table A rows × Table B rows). A 10k-row table paired with a 100k-row table generates 1 billion rows, causing out-of-memory crashes.

Trap 4: The IN vs EXISTS subquery execution

Interviewer: "Is the `EXISTS` operator always faster than the `IN` operator for subquery filters?"

Deflection: `EXISTS` is typically faster for subqueries because it short-circuits (returns `TRUE` upon finding the first match). `IN` evaluates all subquery results first. However, modern planners optimize both to similar plans.

Trap 5: The Primary Key vs Clustered Index myth

Interviewer: "Does creating a Primary Key always create a Clustered Index?"

Deflection: Usually yes by default, but they are conceptually separate. A primary key is a logical unique constraint; a clustered index is a physical layout organization. You can specify a non-clustered primary key.



SQL Common Traps (Part 2)

Tactics to Evade Trick Interview Questions with Confident Answers

Trap 6: The CTE Materialization Trap

Interviewer: "Are Common Table Expressions (CTEs) always written to disk memory?"

Deflection: No. CTEs are logical wrappers compiled inline by query optimizers (like virtual views). Only temporary tables physically write data to disk structures.

Trap 7: Indexing all columns to speed queries

Interviewer: "Why don't we index every single column in our database tables?"

Deflection: Because indexes add write overhead. Every write operation (INSERT/UPDATE/DELETE) must physically update B-Tree structures, degrading write performance and increasing disk storage footprint.

Trap 8: The Serializable isolation overhead

Interviewer: "Is Serializable always the best isolation level because it guarantees correctness?"

Deflection: While safest for consistency, it is the worst for performance. It places extensive lock ranges, causing concurrency bottlenecks, transaction timeouts, and deadlock aborts under high web traffic.

Trap 9: COUNT(col) NULL evaluation

Interviewer: "Does COUNT(column_name) count records that contain NULL?"

Deflection: No. COUNT(column_name) completely excludes NULL values, counting only populated rows. Only COUNT(*) counts all rows including NULL records.

Trap 10: SELECT alias filtering inside WHERE

Interviewer: "Why can you sort by a column alias in ORDER BY, but you cannot filter by it in WHERE?"

Deflection: Because WHERE executes **before** SELECT in the database logical processing sequence, so aliases are not defined yet. ORDER BY executes **after** SELECT, recognizing the aliases.

